

ALLIANCE UNIVERSITY

Private University established in Karnataka State by Act No.34 of year 2010
Recognized by the University Grants Commission (UGC), New Delhi

Database Management Systems

AIRLINE RESERVATION SYSTEM

A Relational Database Design for Flight Booking & Management

Bachelor of Computer Application
Semester – III

R MONISHA BANGERA
2511021240053

Submitted To -

Faculty Name: Renita Joy R

Faculty Signature:

1. Introduction

Every application that stores data must choose a database model. For decades the relational database (RDBMS) was the default. It stores data in tables of rows and columns, links tables with primary and foreign keys, and is queried with SQL. Since the late 2000s, NoSQL ("not only SQL") databases have grown popular because web-scale applications needed to handle huge data volumes, flexible data shapes and many simultaneous users across many servers.

This report compares the two families on scalability, consistency, performance and use cases, and explains why organizations pick different technologies. The running example is the Airline Reservation System designed in the earlier assignments: nine tables (Passenger, Airport, Aircraft, Flight, Seat, Booking, Ticket, Payment, Baggage) joined by foreign keys, with Booking resolving the many-to-many relationship between Passenger and Flight.

2. Overview of the Two Technologies

2.1 Relational databases

A relational database organizes data into tables with a fixed schema defined in advance (the CREATE TABLE statements of the airline design). Data is normalized so each fact is stored once, and relationships are enforced through foreign keys. Transactions follow the **ACID** properties: Atomicity, Consistency, Isolation and Durability. Examples: MySQL, PostgreSQL, Oracle Database, Microsoft SQL Server.

2.2 NoSQL databases

NoSQL databases drop the fixed table model in favour of flexible structures. There are four main types:

Type	How data is stored	Examples	Typical use
Document	JSON-like documents with nested fields	MongoDB, Couchbase	Catalogs, user profiles, content
Key-value	A unique key mapped to a value	Redis, Amazon DynamoDB	Caching, sessions, shopping carts
Column-family (wide-column)	Rows with flexible columns grouped into families	Apache Cassandra, HBase	Time-series, logs, very large write loads
Graph	Nodes and edges	Neo4j, Amazon Neptune	Social networks, fraud detection, route networks

Many NoSQL systems favour the **BASE** model: Basically Available, Soft state, Eventually consistent. They accept that copies of data on different servers may briefly disagree in exchange for speed and availability.

3. Comparison Criteria

3.1 Scalability

Relational: Traditionally scales *vertically* (scale-up): a bigger server with more CPU, memory and faster disks. This is simple but has a hardware and cost ceiling. Read load can be spread with read replicas, and large tables can be partitioned or sharded, but sharding a relational database is complex because joins and transactions across shards are hard. Distributed SQL systems (for example Google Spanner, CockroachDB) narrow this gap but are newer and more specialized.

NoSQL: Designed to scale *horizontally* (scale-out): add more ordinary servers and the database spreads data across them automatically (sharding) and keeps copies (replication). Cassandra and DynamoDB are built around this idea, so capacity grows almost linearly with the number of nodes.

3.2 Consistency

Relational: Provides strong consistency through ACID transactions. After a transaction commits, every user sees the same data. Constraints (PRIMARY KEY, UNIQUE, NOT NULL, FOREIGN KEY, CHECK) are enforced by the database itself.

NoSQL: Often offers eventual consistency. The **CAP theorem** states that when a network partition occurs, a distributed system must choose between consistency and availability. Many NoSQL systems choose availability, so a read may briefly return stale data. Many products let the designer tune this per query (Cassandra consistency levels, DynamoDB strongly consistent reads), and some, such as MongoDB, now support multi-document ACID transactions, though with more overhead than a traditional RDBMS.

3.3 Performance

Relational: Excellent for complex queries. Joins, aggregation, GROUP BY and ad-hoc reporting are optimized by mature query planners and indexes. Performance can drop when a query joins many large tables, or when a single server becomes the write bottleneck.

NoSQL: Excellent for simple access patterns at very high volume: read or write one key or one document in milliseconds. Data is usually denormalized so that everything needed for a request sits in one record and no join is needed. The trade-off is that queries not planned for at design time (cross-collection reports, ad-hoc joins) are slow or awkward.

3.4 Data model flexibility

Relational schemas are rigid: adding a column means altering the table, but this rigidity protects data quality. NoSQL schemas are flexible ("schema-on-read"): different documents can have different fields, which suits fast-changing or varied data but moves the responsibility for validation to the application.

3.5 Summary table

Criterion	Relational (RDBMS)	NoSQL
Data model	Tables, rows, columns; fixed schema	Document, key-value, column-family, graph; flexible schema
Query language	SQL (standard)	Varies by product (MongoDB queries, CQL, Cypher, APIs)

Criterion	Relational (RDBMS)	NoSQL
Scalability	Mainly vertical; horizontal is complex	Designed for horizontal scale-out
Consistency	Strong (ACID)	Often eventual (BASE); tunable in many products
Joins and relationships	Native, enforced by foreign keys	Limited or none; data is denormalized
Performance strength	Complex queries, aggregation, reporting	Fast simple reads and writes at massive volume
Data redundancy	Minimized through normalization	Accepted deliberately for speed
Integrity enforcement	Done by the database	Mostly done by the application
Best for	Structured, transactional, relationship-heavy data	Large, fast-growing, varied or real-time data

4. Applying the Comparison to the Airline Reservation System

The airline schema shows clearly why the choice matters.

Bookings, payments and tickets need a relational database. A seat on a flight can be sold only once. In the relational design this is protected by a transaction and a UNIQUE constraint on (Flight_ID, Seat_ID) for active bookings: if two customers try to buy seat 12A at the same moment, the database lets one commit and rejects the other. Foreign keys guarantee that every Payment and Ticket belongs to a real Booking, and a single transaction can create the booking, record the payment and issue the ticket together or not at all. With eventual consistency, two servers could each tell a customer that 12A is free, which creates the double booking problem in the analysis of the current schema. **Flight search and caching suit NoSQL.** Millions of travellers search flights far more often than they book. Search results can be cached in a key-value store such as Redis, so repeated searches answer in milliseconds without loading the main database. Short-lived data such as login sessions and a half-completed booking cart also fits key-value storage. **Logs and activity data suit wide-column or document stores.** Clickstream, search history, mobile app events and flight-status updates arrive in huge volume and are written once and read later. A system such as Cassandra can absorb these writes across many

servers. Flexible passenger profile or preference data (meal choice, loyalty details, device information) fits a document database such as MongoDB.

Route networks suit graph databases. Finding multi-stop itineraries, connecting airports through hubs, or detecting suspicious booking rings is a graph problem: airports are nodes and flights are edges. A graph database such as Neo4j answers these questions more naturally than recursive SQL joins.

Airline function	Best fit	Reason
Booking, seat allocation, payment, ticketing	Relational (MySQL / PostgreSQL)	ACID transactions, constraints, no double booking
Passenger master data, flights, aircraft, airports	Relational	Structured, stable, relationship-heavy
Flight-search result cache, sessions	Key-value (Redis)	Very fast reads, temporary data
Search history, app events, logs	Wide-column / document (Cassandra, MongoDB)	Massive write volume, flexible fields
Multi-stop route finding, fraud rings	Graph (Neo4j)	Relationship traversal

5. Why Organizations Choose Different Technologies

No database is best for everything. Organizations choose based on requirements:

- **Data correctness requirements.** Banks, airlines (for payments and seats) and hospitals cannot tolerate lost or conflicting updates, so they use relational databases with ACID guarantees.
- **Scale and growth.** A company serving hundreds of millions of users, with data volume that doubles often, needs a database that scales out on commodity hardware.
- **Data shape.** Highly structured, uniform data fits tables. Varied, nested or fast-changing data fits documents. Highly connected data fits graphs.
- **Query patterns.** If the business needs complex reports and ad-hoc analysis, SQL is a major advantage. If it only needs fast lookups by key, NoSQL is simpler and faster.

- **Availability needs.** Services that must stay up even during network failures (social feeds, catalogs) may accept slightly stale data in return for availability.
- **Team skills and ecosystem.** SQL is a widely taught standard with mature tools, backup practice and a large talent pool. NoSQL products each have their own query model and operational practices.
- **Cost and speed of development.** A flexible schema speeds early prototyping, while a relational schema saves effort later by enforcing quality.

Because requirements differ across parts of one application, many organizations use **polyglot persistence**: several databases, each for the job it does best. Example: relational for transactions, a key-value cache for speed, a document store for profiles, and a search engine or graph database for specialized queries.

6. Real-World Examples

Organization / domain	Technology	Why
Banks and payment systems	Relational (Oracle, PostgreSQL, SQL Server)	Money transfers must be atomic and strongly consistent
Airline and hotel booking cores	Relational	Inventory (seats, rooms) must never be oversold
Netflix	Apache Cassandra for large parts of its data	Huge volume, spread across regions, high availability
Amazon (shopping cart and many services)	DynamoDB, which grew from the Dynamo design built for always-available carts	Must accept writes even during failures; scales automatically
Online catalogs and content platforms	MongoDB (document)	Products have different attributes; flexible schema
Web applications at high traffic	Redis (key-value cache)	Sub-millisecond reads, session storage, rate limiting
Social networks and fraud detection	Graph databases (Neo4j and similar)	Relationships between people, accounts or devices are the main data

Organization / domain	Technology	Why
ERP and accounting software	Relational	Complex reporting and strict data integrity

Large companies commonly combine both families. An e-commerce company, for example, keeps orders and payments in a relational database, the product catalog in a document store, and sessions and caches in Redis.

7. Advantages and Limitations

Relational databases

- Advantages: strong consistency, powerful SQL and joins, enforced integrity, mature tools, little data redundancy when normalized.
- Limitations: harder to scale horizontally, rigid schema changes, can be slower for extremely high-volume simple workloads, poor fit for unstructured data.

NoSQL databases

- Advantages: easy horizontal scaling, high availability, flexible schema, very fast simple reads and writes, suits big data and real-time workloads.
- Limitations: weaker or tunable consistency, limited joins, redundancy from denormalization, no single standard query language, more integrity logic in the application.

8. Conclusion

Relational and NoSQL databases solve different problems. Relational databases give strong consistency, enforced relationships and rich querying, which is why the core of the Airline Reservation System (bookings, seats, payments and tickets) belongs in a relational design, and why constraints such as a unique seat per flight are needed to prevent the anomalies identified in the design analysis. NoSQL databases give effortless horizontal scaling, flexibility and speed, which makes them a strong choice for caching, logs, user activity and route analysis around that core.

The practical lesson is that organizations should choose by requirement, not by trend: correctness and complex queries point to relational, while massive scale, flexible data and high availability point to NoSQL. Most large systems, including a modern airline platform, use both together.